Capstone: Build a Low-Latency Order Book

Level: Intermediate C++
Duration: 2 Weeks .. Submit by 17th October 2026 11:59PM
Allowed to do in teams of 2 (2 is the max size :))
Language: C++17 or later

---

1. What Is This About?

An **order book** is a table where buy orders (called **bids**) and sell orders (called **asks**) wait.

- **Best Bid:** The highest buy price.
- **Best Ask:** The lowest sell price.
- **BBO (Best Bid and Offer):** The best bid and best ask together.

Trading systems may need to read the BBO millions of times per second. Therefore, the order book needs to be extremely efficient.

Your task is to:

1. Build an order book for a single instrument.
2. Implement it using **three different data-structure designs**.
3. Benchmark all three implementations.
4. Profile them using Linux `perf`.
5. Analyze **why** one implementation performs better than another.

You will not be told which data structure to use. The goal is to experiment, measure, and understand the performance characteristics of each design.

---

2. Rules of the Game

The order book has the following constraints:

- **One instrument only.**
- No symbols or multiple stocks.

- **Tick size:** `0.01`
- Prices must always be multiples of `0.01`.
- **Bid price range:** `[50.00, 99.99]`
- **Ask price range:** `[100.01, 150.00]`
- **Mid price:** `100.00`
- There are **10,000 possible price levels** in total.
- There is **no order matching** and no trade generation.
- Orders simply rest in the book until cancelled.
- Multiple orders can exist at the same price.
- The quantity at a price level is the **sum of all order quantities at that price**.
- Order IDs are unique and are never reused.
- The implementation must be **single-threaded**.
- Use the **C++ standard library only**. No Boost or external libraries for the order book itself.
- The program must compile with:

-O2

---

3. Required API

Your order book must support the following five operations:

```
enum class Side { Buy, Sell };

struct Level {
    double price;
    long qty;
};

struct BBO {
    bool has_bid;
    double bid_price;
    long bid_qty;

    bool has_ask;
    double ask_price;
    long ask_qty;
};

class OrderBook {
public:
```

```
    void add_order(uint64_t id, Side side, double price, long qty);

    bool cancel_order(uint64_t id);

    BBO get_bbo() const;

    std::vector<Level> get_bids(int depth = 10) const;

    std::vector<Level> get_asks(int depth = 10) const;
};
```

You may modify the interface if needed.

For example, you may use **integer ticks instead of `double` prices**.

If you change the interface, clearly document the changes in your README.

Required Behavior

Cancel Unknown Order

Cancelling an order that:

- does not exist, or
- has already been cancelled

must return:

false

The program must not crash.

Empty Book

Calling `get_bbo()` on an empty book should indicate:

has_bid == false
has_ask == false

Insufficient Depth

If the book contains only three price levels and the caller requests:

get_bids(10)

return the available three levels.

---

4. The Three Implementations

This is the main part of the assignment.

You must implement the order book **three different times**, using three genuinely different underlying data structures.

Implementation A

You may use a standard container such as:

- `std::map`
- `std::priority_queue`
- or another suitable standard data structure.

Implementation B

Use a different data-structure approach from Implementation A.

Implementation C — Your Own Design

The third implementation must be **your own design**.

Possible ideas include:

- Fixed-size array
- Vector
- Hash map + array
- Price-indexed structure
- Combination of multiple data structures
- Any other design you believe is suitable

Do not simply rename the same implementation three times.

The three designs must genuinely differ in how they:

- Store orders
- Store price levels
- Find the best price
- Locate orders for cancellation

Required Analysis

For each implementation, document:

1. What data structure(s) you used.
2. How orders are stored.
3. How price levels are stored.
4. How the best bid/ask is found.
5. How an order is found during cancellation.
6. Big-O complexity of:

Operation	Complexity
`add_order`	?
`cancel_order`	?
`get_bbo`	?
`get_bids`	?
`get_asks`	?

---

5. Correctness Check

Before benchmarking performance, you must prove that all three implementations produce the same results.

Run the **exact same random workload** through all three implementations.

For every operation, verify that:

- BBO results match.
- Bid levels match.
- Ask levels match.
- Quantities match.
- Price levels match.

You may implement this as:

./bench --verify

or as a separate test program.

The correctness test must automatically report whether all three implementations agree.

Example:

Running correctness test...

Implementation A: PASS
Implementation B: PASS
Implementation C: PASS

All implementations produce identical results.

---

6. Benchmark

Create **one benchmark program** that allows the implementation to be selected from the command line.

Example:

./bench --impl A
./bench --impl B
./bench --impl C

Or can use

“perf”

Do not create three separate benchmark programs.

The same workload must be used for all three implementations.

Workload

Run a total of:

1,000,000 operations

Use a fixed random seed so that the workload is repeatable.

For example:

std::mt19937 rng(42);

Operation Distribution

Approximately:

- **70% Add**
- **20% Cancel**
- **10% BBO query**

Additionally:

> Every 100 operations, call `get_bids(10)` and `get_asks(10)`.

Random Parameters

Side:

- 50% Buy
- 50% Sell

Quantity:

1–100

Price:

Random valid tick within the appropriate price band.

Cancel:

Cancels must always select an order that is currently live.

---

7. Benchmark Measurements

Use:

std::chrono::steady_clock

Measure:

- Total execution time
- Average nanoseconds per `add_order`
- Average nanoseconds per `cancel_order`
- Average nanoseconds per `get_bbo`

Measure each operation type separately.

Run each implementation **5 times**.

Report the **median** result.

Preventing Compiler Optimization

Print the final BBO or another checksum at the end of the benchmark so that the compiler cannot optimize away the work.

For example:

Final BBO:
Bid: 99.98 x 450
Ask: 100.01 x 320

Checksum: 928371

---

8. Profiling With `perf`

Performance numbers alone are not enough.

You must investigate **why** one implementation performs better than another.

Use Linux `perf`.

`perf stat`

For each implementation, run:

```
perf stat -r 5 \
-e task-clock,cycles,instructions,branches,branch-misses,\
cache-misses,L1-dcache-load-misses,dTLB-load-misses,page-faults \
```

./bench --impl A

Repeat for:

./bench --impl B
./bench --impl C

If some counters are unavailable on your machine, report the counters that are available and mention which ones were unavailable.

`perf record`

Use:

perf record -g ./bench --impl A
perf report

Repeat for B and C.

Use the results to determine where the program spends its time.

---

9. Understanding the `perf` Counters

Counter	Meaning
`cycles`	Number of CPU clock cycles used
`instructions`	Number of CPU instructions executed
`IPC`	Instructions per cycle
`branches`	Number of branch instructions
`branch-misses`	Branches where the CPU predicted incorrectly
`cache-misses`	Data accesses that missed the relevant cache hierarchy

`L1-dcache-load-misses`	Loads that missed the L1 data cache
`dTLB-load-misses`	Data address translation misses
`page-faults`	Number of page-fault events

IPC

IPC means:

Instructions Per Cycle

A higher IPC generally means the CPU is able to execute more useful instructions per clock cycle.

---

10. Results Table

Fill in the following table in your report:

Metric	Implementation A	Implementation B	Implementation C
Total time (ms)			
ns / add			
ns / cancel			
ns / BBO			
IPC			
Branch misses (%)			
L1-dcache-load-misses			
Cache misses			
Page faults			

---

11. Performance Analysis

Write a **1–2 page report** answering the following questions.

11.1 Which implementation is fastest?

Which implementation is fastest for:

- `add_order`
- `cancel_order`
- `get_bbo`

Why do you think it performs better?

11.2 Branch Prediction

Which implementation has the most branch misses?

Look at the implementation and explain what may cause the CPU to make incorrect branch predictions.

11.3 Cache Behavior

Which implementation has the most cache misses?

Does the implementation:

- Access contiguous memory?
- Jump around memory?
- Follow pointers?
- Allocate many small objects?

Explain why this might affect performance.

11.4 Page Faults

Which implementation causes the most page faults?

What does this tell you about its memory usage or allocation behavior?

11.5 Production Choice

Imagine you are allowed to deploy **only one implementation** to a real trading system.

Which one would you choose?

Explain your decision using:

- Latency
- Predictability
- Memory usage
- Cache behavior
- Complexity
- Maintainability

Do not simply choose the implementation with the lowest average latency.

11.6 What Would You Try Next?

Suggest additional optimizations you would investigate.

Examples:

- Integer ticks instead of floating-point prices
- Fixed-size arrays
- Better memory layout
- Pre-allocation
- Object pools
- Cache-friendly data structures
- Branch reduction
- Better order-ID lookup
- Avoiding unnecessary allocations

---

12. Deliverables

Submit the following:

Source Code

- Three order-book implementations
- Benchmark program
- Correctness test / `--verify` mode

README

Explain:

- Don’t forget to add SST email roll number :) and raise a PR
- How to build the project
- How to run the correctness test
- How to run each benchmark
- Any interface changes
- Your data-structure choices

A single `g++` command or a simple CMake project is sufficient.

Correctness Results

Include output showing that all three implementations agree.

Performance Results

Include:

- `perf stat` output for Implementation A
- `perf stat` output for Implementation B
- `perf stat` output for Implementation C

Report

Include the completed results table and the **1–2 page performance analysis**.

Submission Format

Submit either:

- PR here https://github.com/KnightKnight27/scaler-HFT-C--2028

Deadline: Two weeks from the date of assignment.

---

13. Grading

Category	Points
Correctness — all five APIs work and implementations agree	35
Three genuinely different designs	20
Clean benchmark + proper `perf` data	25
Performance analysis and explanations	15
Code readability	5
Total	**100**

---

14. Hints

Read this section only if you get stuck.

Hint 1 — Cancellation

Cancellation must be fast too.

Ask yourself:

> How will I find an order by its ID?

Think about this before choosing your data structure.

Hint 2 — BBO

Does `get_bbo()` really need to search the entire book every time?

Is there a way to maintain the best bid and best ask so that retrieving them is cheaper?

Hint 3 — Price Representation

The tick size is fixed at `0.01`.

Ask yourself:

> Do I really need floating-point numbers to represent prices?

For example:

100.01 → 10001 ticks
100.02 → 10002 ticks
100.03 → 10003 ticks

Hint 4 — Orders vs Price Levels

There are two different problems:

Per-order storage

Order ID → Order information

Per-price storage

Price → Total quantity

Think carefully about how these two pieces of information should be represented.

Hint 5 — Common Interface

Keep all three implementations behind the same logical interface so that the benchmark can test them using the same workload.

---

15. Rules

You may discuss ideas with your classmates.

However:

- Write your own implementation.
- Do not copy-paste code from the internet.
- You must be able to explain every line of your submission.

- Your benchmark results must come from your own machine/environment.
- Your `perf` analysis must be based on your actual measurements.

---

16. FAQ

Q: Do I need to match orders or generate trades?

No.

This is a resting order book.

Add → order rests
Cancel → order is removed

There is no matching engine.

Q: Can my three implementations share the same benchmark?

Yes.

In fact, they should.

Use one benchmark program:

./bench --impl A
./bench --impl B
./bench --impl C

This ensures that all implementations are tested with the same workload.

Q: Do I need multiple threads?

No.

Keep the entire assignment single-threaded.

Q: `perf` doesn't work on my machine. What should I do?

If you are using macOS, a VM, or a restricted environment, some hardware performance counters may not be available.

Use the lab Linux machines if possible.

If certain counters are unavailable, report the counters that do work and clearly mention the unavailable ones.

Q: What if my own design is the slowest?

That's completely fine.

A slow design with a **good explanation of why it is slow** is better than a fast implementation that you cannot explain.

The goal of this assignment is not simply to find the fastest data structure.

The goal is to understand **why** different designs behave differently on modern CPUs.